Ensemble Learning for Decision Trees
Complementary Course in Statistical Learning and Applications — Master 1 AM UGA

Razan MHANNA

April 2026

Ensemble Learning

Main idea

Instead of relying on a single model, **ensemble learning** combines the predictions of several models in order to obtain a more accurate and more robust predictor.

- A single model may have high variance or limited predictive power
- Combining several models can improve **generalization**
- The gain comes from **diversity** among the base learners

Key principle

A good ensemble requires:

- base learners that are reasonably accurate
- base learners that are not all making the same errors

How an Ensemble Combines Predictions

Assume we train B base learners:

$$f_1(x), f_2(x), \ldots, f_B(x)$$

Regression

The ensemble prediction is usually the average:

$$\hat{f}_{\text{ens}}(x) = \frac{1}{B} \sum_{b=1}^{B} f_b(x)$$

Classification

The ensemble prediction is usually obtained by majority vote:

$$\hat{y}_{\text{ens}}(x) = \mathrm{mode}\big(f_1(x), \ldots, f_B(x)\big)$$

- Averaging reduces variance
- Voting stabilizes the decision

Why Can Ensembles Improve Performance?

Bias–Variance point of view

Ensembles are especially useful when the base learner has **high variance**, such as decision trees.

- A single tree may change a lot if the training set changes slightly
- Averaging many trees reduces this instability
- If the trees are sufficiently **decorrelated**, the variance reduction is stronger

Key message

The success of an ensemble depends not only on the strength of individual learners, but also on their **diversity**.

Two Major Families of Ensembles

Bagging

Models are trained **in parallel** on different resampled versions of the data, then aggregated.

Boosting

Models are trained **sequentially**, each new learner focusing on the errors of the previous ones.

- Bagging mainly reduces variance
- Boosting can reduce both bias and variance, but is more sensitive to tuning

Bagging: Bootstrap Aggregating

Definition

Bagging stands for **bootstrap aggregating**. It builds several predictors on bootstrap samples of the training set and then combines their predictions.

- Draw B bootstrap samples from the original dataset
- Train one base learner on each sample
- Aggregate the predictions by averaging or voting

Main purpose

Bagging is designed to reduce the **variance** of unstable learners.

Bootstrap Sampling

Suppose the training set is

$$\mathcal{D} = \{(x_1, y_1), \ldots, (x_n, y_n)\}.$$

Bootstrap sample

A bootstrap sample is obtained by drawing n observations **with replacement** from $\mathcal{D}$.

- Some observations may appear several times
- Some observations may be absent
- Each bootstrap sample is a slightly different version of the original dataset

Toy example

If the dataset is $\{1, 2, 3, 4, 5\}$, one bootstrap sample could be:

$$\{2, 5, 2, 1, 4\}$$

Bagging Algorithm

Given a training set $\mathcal{D}$ and a base learning algorithm:

1. For $b = 1, \ldots, B$:
   - draw a bootstrap sample $\mathcal{D}^{(b)}$ from $\mathcal{D}$
   - train a base learner f_b on $\mathcal{D}^{(b)}$
2. Aggregate the predictions:

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^{B} f_b(x) \quad \text{(regression)}$$

$$\hat{y}_{\text{bag}}(x) = \mathrm{mode}\big(f_1(x), \ldots, f_B(x)\big) \quad \text{(classification)}$$

Why Are Trees Good Candidates for Bagging?

Unstable learners

A learner is called **unstable** if a small change in the training set can lead to a large change in the fitted model.

- Decision trees are highly unstable
- Their variance is often high
- Bagging is therefore particularly effective for trees

Interpretation

Bagging many deep trees produces a more stable predictor than a single deep tree.

Variance Reduction by Averaging

Let $f_1(x), \ldots, f_B(x)$ be predictors with variance σ^2.

Idealized case

If they were independent, then

$$\mathrm{Var}\left(\frac{1}{B}\sum_{b=1}^{B} f_b(x)\right) = \frac{\sigma^2}{B}.$$

- Averaging reduces variance
- In practice, base learners are correlated
- The more correlated they are, the smaller the gain

Important

This is why diversity among learners is essential.

Out-of-Bag (OOB) Error

Key idea

For each bootstrap sample, some observations are not selected. These are called **out-of-bag** observations.

- A tree can be evaluated on the observations that were not used to train it
- Aggregating these predictions gives an estimate of the test error
- This is called the **OOB error**

Why useful?

OOB error provides an internal estimate of generalization performance without requiring a separate validation set.

Bagging: Advantages and Limitations

Advantages

- Reduces variance
- Improves predictive stability
- Works well with high-variance learners such as trees
- OOB error can be used for evaluation

Limitations

- Does not significantly reduce bias
- Can be less interpretable than a single tree
- If the learners are too correlated, the gain is limited

From Bagging to Random Forests

Observation

Bagging improves trees by averaging them, but many trees may still look similar because the same strong predictors tend to be selected near the top of each tree.

- Trees remain correlated
- Correlation limits the variance reduction

Random Forest idea

In addition to bootstrap sampling, random forests add **random feature selection at each split** in order to decorrelate the trees further.

Random Forest Algorithm

Given a training set $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$:

1. For $b = 1, \ldots, B$:
   - Draw a bootstrap sample $\mathcal{D}^{(b)}$ of size n
   - Grow a decision tree on $\mathcal{D}^{(b)}$
   - At each node:
     - randomly choose m predictors among the p available predictors
     - find the best split using only these m predictors
2. Aggregate the predictions of all trees

Prediction rule

Regression:

$$\hat{f}(x) = \frac{1}{B} \sum_{b=1}^{B} f_b(x)$$

Classification:

$$\hat{y}(x) = \mathrm{mode}\big(f_1(x), \ldots, f_B(x)\big)$$

Why Does Random Feature Selection Help?

- Averaging reduces variance only if the trees are not too correlated
- If all trees are very similar, averaging brings limited improvement
- Randomly restricting the candidate variables at each split creates more diversity

Variance intuition

If the individual tree predictors have variance σ^2 and pairwise correlation ρ, then the variance of the average behaves roughly like:

$$\mathrm{Var}\left(\frac{1}{B}\sum_{b=1}^{B} f_b(x)\right) \approx \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$$

Interpretation

Reducing correlation ρ is crucial. This is exactly what random forests are designed to do.

Choice of the Number of Candidate Predictors m

At each split, a random forest considers only m predictors out of the full set of p predictors.

Typical defaults

- Classification: $m \approx \sqrt{p}$
- Regression: $m \approx p/3$

- Small m:
  - more randomness
  - lower correlation between trees
  - possibly weaker individual trees
- Large m:
  - trees become stronger
  - but more correlated

Trade-off

We must balance individual tree strength and diversity across trees.

Out-of-Bag (OOB) Error

Observation

Each tree is trained on a bootstrap sample, so some observations are not used to train that tree. These are called **out-of-bag** observations.

- For each training sample, we can collect predictions from trees that did not see it
- Comparing these predictions to the true label gives an estimate of prediction error

OOB error

$$\text{OOB error} = \frac{1}{n} \sum_{i=1}^{n} \mathbf{1}\left(\hat{y}_i^{\text{OOB}} \neq y_i\right)$$

Why useful?

OOB error provides an internal estimate of generalization performance without a separate validation set.

Random Forest for Classification and Regression

Classification

Each tree predicts a class label. The forest prediction is obtained by majority vote.

Regression

Each tree predicts a numerical value. The forest prediction is obtained by averaging.

- In both cases, the principle is the same:
  - many randomized trees
  - aggregation at the end
- Random forests are flexible and handle nonlinear relationships well

Variable Importance in Random Forests

Random forests can provide measures of feature importance.

Two common notions

- **Mean decrease in impurity**: total impurity reduction contributed by a variable across the forest
- **Permutation importance**: performance drop after randomly permuting one feature

Caution

Feature importance helps interpretation, but it does not imply causality. Correlated predictors can also make importance scores harder to interpret.

Main Hyperparameters of a Random Forest

- `n_estimators`: number of trees
- `max_features`: number of candidate predictors at each split
- `max_depth`: maximum depth of each tree
- `min_samples_split`: minimum number of samples to split a node
- `min_samples_leaf`: minimum number of samples in a leaf

Practical interpretation

- More trees usually improve stability
- Shallower trees may reduce overfitting
- Smaller `max_features` increases diversity

Advantages and Limitations of Random Forests

Advantages

- Strong predictive performance
- Handles nonlinearities and interactions automatically
- Robust to overfitting compared with a single tree
- Works with high-dimensional data
- Provides OOB error and variable importance

Limitations

- Less interpretable than a single tree
- Can be computationally expensive
- Large forests may be memory-intensive
- Feature importance can be misleading when predictors are correlated

Python Example: Random Forest Classification

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import
    train_test_split
from sklearn.ensemble import
    RandomForestClassifier
from sklearn.metrics import accuracy_score

data = load_breast_cancer()
X, y = data.data, data.target

X_train, X_test, y_train, y_test =
    train_test_split(
     X, y, test_size=0.25, random_state=0,
         stratify=y
)

rf = RandomForestClassifier(
     n_estimators=200,
     max_features="sqrt",
     random_state=0
```

Python Example: Out-of-Bag Score

```
from sklearn.ensemble import
    RandomForestClassifier

rf = RandomForestClassifier(
     n_estimators=300,
     max_features="sqrt",
     oob_score=True,
     random_state=0
)

rf.fit(X_train, y_train)

print("OOB score:", rf.oob_score_)
```

- oob_score_ gives an internal estimate of accuracy
- Useful when you want a quick validation estimate

Python Example: Feature Importance

```
import pandas as pd
import matplotlib.pyplot as plt

importances = rf.feature_importances_
names = data.feature_names

imp_df = pd.DataFrame({
    "feature": names,
    "importance": importances
}).sort_values("importance", ascending=False)

print(imp_df.head(10))

plt.figure(figsize=(8,5))
plt.barh(imp_df["feature"][:10], imp_df["
   importance"][:10])
plt.gca().invert_yaxis()
plt.title("Top 10 feature importances")
plt.show()
```

Python Example: Cross-Validation

```
from sklearn.model_selection import
    cross_val_score, StratifiedKFold

cv = StratifiedKFold(n_splits=5, shuffle=True,
    random_state=0)

rf = RandomForestClassifier(
     n_estimators=200,
     max_features="sqrt",
     random_state=0
)

scores = cross_val_score(rf, X, y, cv=cv,
    scoring="accuracy")

print("CV scores:", scores)
print("Mean accuracy:", scores.mean())
print("Std:", scores.std())
```

Python Example: Simple Hyperparameter Tuning

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    "n_estimators": [100, 200, 500],
    "max_features": ["sqrt", "log2", None],
    "max_depth": [None, 3, 5, 10]
}

rf = RandomForestClassifier(random_state=0)

grid = GridSearchCV(
    rf,
    param_grid=param_grid,
    cv=5,
    scoring="accuracy"
)

grid.fit(X_train, y_train)

print("Best parameters:", grid.best_params_)
```

Practical Comments

- Random forests are often strong baseline models
- They require much less preprocessing than SVM or logistic regression
- They are particularly useful when:
  - interactions are important
  - the decision boundary is nonlinear
  - interpretability is less critical than predictive performance

In practice

Before moving to more complex methods such as boosting or deep learning, a random forest is often a very strong benchmark.

Boosting: Main Idea

Definition

Boosting is an ensemble method that builds a strong predictor by combining many **weak learners** trained **sequentially**.

- Unlike bagging, the models are **not trained independently**
- Each new learner focuses on what the previous learners did poorly
- The final predictor is a weighted combination of weak learners

Key principle

Boosting improves performance by **correcting previous errors step by step**.

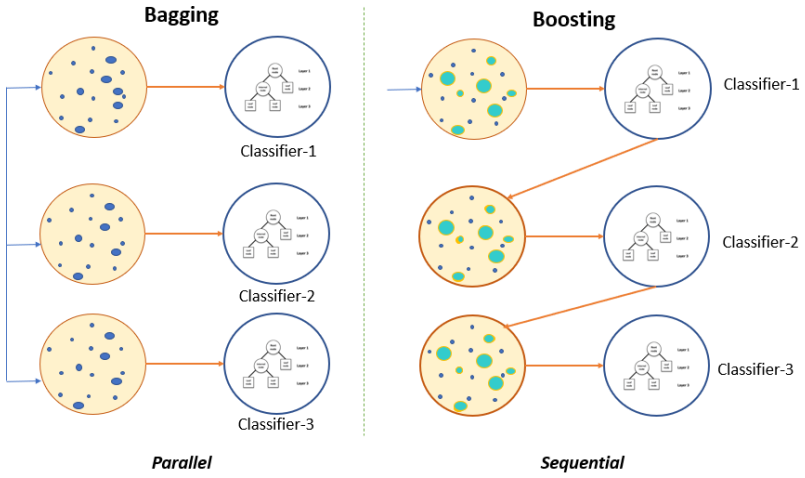
Bagging vs Boosting

Bagging

- learners are trained in parallel
- each learner sees a bootstrap version of the data
- aggregation mainly reduces variance

Boosting

- learners are trained sequentially
- each learner focuses on the remaining errors
- aggregation can reduce both bias and variance



General Boosting Formulation

We build an additive model of the form:

$$F_M(x) = \sum_{m=1}^{M} \gamma_m h_m(x)$$

where:

- $h_m(x)$ is the m-th weak learner
- γ_m is its weight
- M is the number of boosting iterations

Interpretation

At each iteration, we add a new weak learner that improves the current predictor:

$$F_m(x) = F_{m-1}(x) + \gamma_m h_m(x)$$

Key idea

Boosting builds the final model **stage by stage**.

What Is a Weak Learner?

Weak learner

A weak learner is a simple model that performs only slightly better than random guessing.

- In boosting, weak learners are often:
  - decision stumps
  - shallow decision trees
- A single weak learner is not very accurate
- But a sequence of weak learners can produce a strong model

Why shallow trees?

They are simple, fast, and allow the boosting procedure to gradually improve the fit.

AdaBoost: Intuition

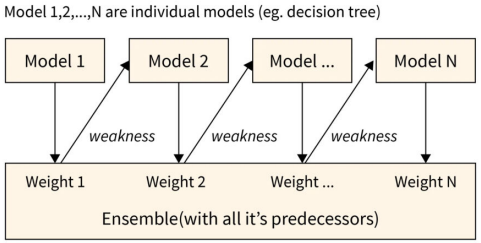
Main idea

AdaBoost increases the importance of observations that were previously misclassified.

- Start with equal weights on all training samples
- Train a weak learner
- Increase the weights of misclassified observations
- Train the next learner on this reweighted dataset

Interpretation

Hard examples receive more attention in later iterations.



AdaBoost: Main Equations

Let $w_i^{(m)}$ be the weight of observation i at iteration m.

1. Train weak learner h_m using weights $w_i^{(m)}$
2. Compute weighted error:

$$\varepsilon_m = \frac{\sum_{i=1}^{n} w_i^{(m)} \mathbf{1}(y_i \neq h_m(x_i))}{\sum_{i=1}^{n} w_i^{(m)}}$$

3. Compute learner weight:

$$\alpha_m = \frac{1}{2} \log \left(\frac{1 - \varepsilon_m}{\varepsilon_m} \right)$$

4. Update sample weights:

$$w_i^{(m+1)} = w_i^{(m)} \exp\big(- \alpha_m y_i h_m(x_i) \big)$$

AdaBoost: Final Prediction

After M iterations, the final classifier is:

$$F_M(x) = \sum_{m=1}^{M} \alpha_m h_m(x)$$

The final prediction is:

$$\hat{y}(x) = \operatorname{sign}\big(F_M(x)\big)$$

Interpretation

- better weak learners get larger weights
- the final decision is a weighted vote

Limitation

AdaBoost is elegant and important historically, but modern boosting methods such as gradient boosting and XGBoost are usually more flexible and more powerful.

Gradient Boosting: Core Idea

We seek a predictor $F(x)$ minimizing:

$$\sum_{i=1}^{n} L\big(y_i, F(x_i)\big)$$

At iteration m, we update:

$$F_m(x) = F_{m-1}(x) + \nu \gamma_m h_m(x)$$

where:

- h_m is a new weak learner
- γ_m is its step size
- $\nu \in (0, 1]$ is the learning rate

Main principle

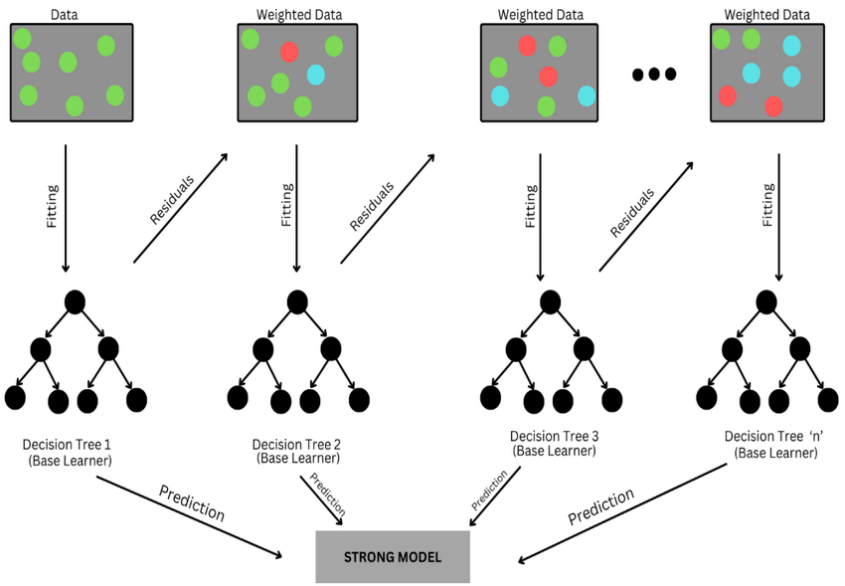
At each step, the new learner is fitted to the **negative gradient** of the loss.

Gradient Boosting: Pseudo-Residuals

At iteration m, for each observation i, define the pseudo-residual:

$$r_{im} = -\left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right]_{F=F_{m-1}}$$

- Compute the negative gradient of the loss
- Fit a weak learner $h_m(x)$ to these pseudo-residuals
- Add the learner to the current model



Gradient Boosting with Squared Error

For regression with squared loss:

$$L(y, F(x)) = (y - F(x))^2$$

the pseudo-residuals become:

$$r_{im} = y_i - F_{m-1}(x_i)$$

Meaning

For squared loss, gradient boosting simply fits the new learner to the ordinary residuals.

- Start with a constant predictor
- Fit a tree to the residuals
- Update the model
- Repeat

Why Can Boosting Work So Well?

- Each weak learner makes a small correction
- The additive model becomes progressively richer
- Shallow trees capture nonlinearities and interactions
- The learning rate regularizes the procedure

Important bias–variance view

- boosting often reduces bias strongly
- but if overused, it can also overfit

Practical message

Boosting can achieve very strong predictive performance, but it must be tuned carefully.

Main Hyperparameters in Boosting

- `n_estimators`: number of boosting iterations
- `learning_rate`: contribution of each new learner
- `max_depth`: depth of each tree
- `subsample`: fraction of training data used at each iteration

Typical effects

- larger `n_estimators`: more flexible model
- smaller `learning_rate`: slower but often better learning
- deeper trees: capture more interactions, but increase complexity

Key trade-off

A small learning rate often requires more trees.

XGBoost

Definition

XGBoost stands for **Extreme Gradient Boosting**. It is an optimized and regularized implementation of gradient boosting.

- efficient and scalable
- supports regularization
- handles missing values
- widely used in tabular-data competitions and applications

Main idea

XGBoost improves gradient boosting with better optimization, regularization, and computational efficiency.

XGBoost Objective Function

At iteration t, XGBoost minimizes an objective of the form:

$$\mathcal{L}^{(t)} = \sum_{i=1}^{n} l\left(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)\right) + \Omega(f_t)$$

where the regularization term is:

$$\Omega(f) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2$$

- T: number of leaves
- w_j: score of leaf j
- γ: penalty on the number of leaves
- λ: penalty on leaf weights

Interpretation

XGBoost controls model complexity explicitly.

Why Is XGBoost So Effective?

XGBoost uses a second-order Taylor approximation of the loss:

$$\mathcal{L}^{(t)} \approx \sum_{i=1}^{n} \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2 \right] + \Omega(f_t)$$

where:

$$g_i = \partial_{\hat{y}^{(t-1)}} l(y_i, \hat{y}^{(t-1)}), \qquad h_i = \partial^2_{\hat{y}^{(t-1)}} l(y_i, \hat{y}^{(t-1)})$$

- g_i: first derivative
- h_i: second derivative

Benefit

This leads to efficient tree construction and optimized split selection.

LightGBM

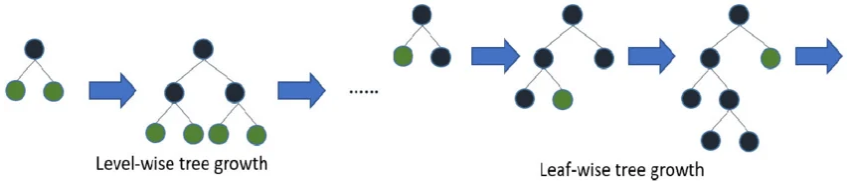
Definition

LightGBM is a gradient boosting framework designed for speed and scalability on large datasets.

- histogram-based split finding
- efficient memory usage
- leaf-wise tree growth
- often very fast on large tabular datasets

Specificity

Unlike level-wise growth, LightGBM grows trees **leaf-wise**, choosing the leaf that gives the largest reduction in loss.



CatBoost

Definition

CatBoost is a boosting algorithm particularly designed to handle categorical features effectively.

- strong default performance
- powerful treatment of categorical variables
- reduced target leakage through ordered boosting

Why important?

Many real tabular datasets contain categorical variables, and naive encoding may lead to poor performance or leakage.

CatBoost strength

CatBoost often performs very well with minimal preprocessing.

XGBoost, LightGBM, CatBoost: Comparison

Method	Main strength	Typical use	Specificity
XGBoost	Strong regularized boosting	General tabular data	Very widely use
LightGBM	Speed and scalability	Large datasets	Leaf-wise growt
CatBoost	Categorical features	Mixed tabular data	Ordered boosting

Practical view

All three are gradient-boosted tree methods, but they differ in optimization strategy and practical strengths.

Python Example: Gradient Boosting in scikit-learn

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import
    train_test_split
from sklearn.ensemble import
    GradientBoostingClassifier
from sklearn.metrics import accuracy_score

data = load_breast_cancer()
X, y = data.data, data.target

X_train, X_test, y_train, y_test =
    train_test_split(
     X, y, test_size=0.25, random_state=0,
         stratify=y
)

gb = GradientBoostingClassifier(
     n_estimators=200,
     learning_rate=0.05,
     max_depth=2,
```

Python Example: XGBoost

```
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score

xgb = XGBClassifier(
    n_estimators=300,
    learning_rate=0.05,
    max_depth=3,
    subsample=0.8,
    colsample_bytree=0.8,
    eval_metric="logloss",
    random_state=0
)

xgb.fit(X_train, y_train)
y_pred = xgb.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred
    ))
```

Python Example: LightGBM

```
from lightgbm import LGBMClassifier
from sklearn.metrics import accuracy_score

lgbm = LGBMClassifier(
    n_estimators=300,
    learning_rate=0.05,
    max_depth=-1,
    num_leaves=31,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=0
)

lgbm.fit(X_train, y_train)
y_pred = lgbm.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred
    ))
```

Python Example: CatBoost

```
from catboost import CatBoostClassifier
from sklearn.metrics import accuracy_score

cat = CatBoostClassifier(
    iterations=300,
    learning_rate=0.05,
    depth=4,
    verbose=0,
    random_state=0
)

cat.fit(X_train, y_train)
y_pred = cat.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred
    ))
```

Python Example: Comparing Boosting Models with CV

```
from sklearn.model_selection import
    cross_val_score, StratifiedKFold

cv = StratifiedKFold(n_splits=5, shuffle=True,
    random_state=0)

models = {
    "XGBoost": XGBClassifier(
        n_estimators=200, learning_rate=0.05,
        max_depth=3, eval_metric="logloss",
            random_state=0
    ),
    "LightGBM": LGBMClassifier(
        n_estimators=200, learning_rate=0.05,
        num_leaves=31, random_state=0
    ),
    "CatBoost": CatBoostClassifier(
        iterations=200, learning_rate=0.05,
        depth=4, verbose=0, random_state=0
    )
```

Practical Tuning Advice for Boosting

- Start with shallow trees
- Use a small learning rate
- Increase the number of estimators if needed
- Monitor validation performance carefully

Common tuning parameters

- learning rate
- number of trees
- tree depth
- subsampling
- column sampling

Warning

Boosting can overfit if the trees are too deep or if too many iterations are used without regularization.

Boosting: Advantages and Limitations

Advantages

- strong predictive performance
- handles nonlinearities and interactions
- often state-of-the-art on tabular data
- flexible loss functions

Limitations

- more sensitive to hyperparameter tuning
- can overfit if not regularized properly
- less interpretable than a single tree
- training is sequential, so less parallel than bagging

Boosting Before Deep Learning

- On structured tabular data, boosted trees are often extremely strong baselines
- They often outperform deep neural networks on small or medium-sized tabular datasets
- This is why boosting remains essential in practical machine learning

Take-home message

Before using a deep neural network on tabular data, it is often wise to compare against boosting methods such as XGBoost, LightGBM, or CatBoost.